

CONTINUUM

A First-Principles Operating System

Design Constitution • Complete Roadmap • Working Agreement

This document is the single source of truth for the Continuum project.
It contains the locked Constitution, the full development roadmap,
and the agreement for how we will build it together.

Version 1.0 — October 2026

PART 1 — THE CONSTITUTION

These are the non-negotiable rules of Continuum. Every implementation decision must conform to them.

1. Core Primitives

The system is built from only these primitives:

- **Objects** — Persistent, append-only, fully versioned entities.
- **Units** — Isolated execution entities that hold code, private memory, and Capabilities.
- **Capabilities** — Unforgeable, transferable tokens of authority.
- **Messages** — The only means of communication between Units.
- **The Nucleus** — The minimal privileged core.

No other fundamental concepts are permitted at the foundation.

2. The Nucleus

The Nucleus is tiny, immutable at runtime, and paranoid. It may only: schedule Units preemptively, create and enforce memory arenas, create/validate/transfer/revoke Capabilities, route Messages, handle basic interrupts, freeze or kill Units, and birth the initial boot Units. Everything else must be implemented as ordinary Units. The Nucleus deliberately drops nearly all power immediately after bootstrap.

3. Objects and Persistence (Unified Persistent Continuum)

- Every change to an Object appends a new version. Objects are defined by their entire history.
- There is no “save” operation and no “unsaved” state.
- Once a version has been acknowledged, it is durable.
- Retention follows tiered policy: recent versions at full fidelity; bookmarks/pins are never auto-removed; older history may be semantically compacted.
- True ephemeral Objects are forbidden at the foundation. Everything is persistent by default.
- The physical split between fast memory and long-term storage is hidden from Units and humans.

4. Units and Execution

- Units are reactive by default: they sleep at zero cost until a Message arrives.
- Each Unit runs in a private memory arena and can only access resources for which it holds Capabilities.
- Units may be frozen, killed, restarted, or replaced without bringing down the system.
- Drivers, services, naming, intent resolution, human interfaces, and Improvers are all ordinary Units.

5. Capabilities and Security

- There is no ambient authority. Authority exists only as explicit Capabilities.
- A Unit can only act on what it has been given.
- Discovery of Objects or names without a relevant Capability is impossible.
- Capabilities can be temporary or long-lived; temporary is preferred when sufficient.
- Humans (and Improvers) can grant, view, and revoke Capabilities. Granting is designed to be low-friction through intent-time proposals and reusable Profiles.

6. Naming and Intent

- Both exact names and rich descriptive phrases are first-class.
- Ambiguity is resolved by presenting ranked suggestions for confirmation.
- Naming and full intent resolution are handled by ordinary Units that can evolve.
- Full intent resolution (“make this photo brighter and send it to David”) is supported from early stages.

7. Self-Healing and Self-Improvement

- The system detects invariant violations, crashes, and capability abuses.
- Faulty Units are immediately contained.
- Recovery uses version history and clean restarts.
- Improver Units may autonomously diagnose failures, propose improved versions of Units, and mark better versions as preferred.
- Improvers cannot modify the Nucleus or grant themselves arbitrary power.
- A Guardian Capability exists as an ultimate human (or hardware-rooted) escape hatch.
- All improvements remain versioned and reversible.

8. Human Interaction & Communication

- The system should feel like a calm, competent colleague with perfect memory.
- Transparency over magic: the human can always inspect Objects, Units, Capabilities, and Messages.
- Console Unit: Clean textual interface, always available.
- Studio Unit: Spatial continuous canvas of living Objects and Verbs (primary long-term interface).
- Full intent loop: express → resolve → propose capabilities → confirm → execute → inspectable result.
- Both interfaces are ordinary Units and may evolve.

9. Bootstrap

The Nucleus starts with minimal power, births a small ordered set of essential Units (Object store, Naming/Intent, Console/Studio, Watcher, initial Improver), then drops nearly all remaining privilege. From that point forward the system runs almost entirely as ordinary Units.

10. Non-Negotiable Invariants

1. The Nucleus remains tiny and runtime-immutable.
2. No ambient authority shall exist.
3. All lasting state is append-only and versioned.
4. Communication occurs only through Messages carrying explicit Capabilities.
5. Units are isolated by default.
6. The system must remain comprehensible in principle to a single skilled human mind.
7. Nothing the human has been told exists may be silently lost.
8. Self-improvement may never compromise the above rules.

11. Explicitly Forbidden at the Foundation

- Hierarchical file systems as a core concept
- Application containers that trap data
- Save buttons or unsaved states
- Ambient/global authority
- Unrestricted shared mutable state
- Self-modification of the Nucleus
- True ephemeral Objects as a foundation primitive

PART 2 — COMPLETE ROADMAP

This is the high-level plan for building Continuum. We move phase by phase. Inside each phase we work in very small steps.

Phase 0 — Design Stabilization (COMPLETED)

We resolved the major open questions, locked the Constitution, and defined how we will work together. This phase is finished.

Phase 1 — Pure Simulation (CURRENT)

Build a pure software simulation that obeys the Constitution. Goal: prove the model feels right and catch design issues early. We will implement this together in small steps.

Phase 1 sub-steps:

- 1.1 Project structure + Constitution file
- 1.2 Capability (unforgeable token)
- 1.3 Message
- 1.4 Object + ObjectStore (append-only versioning)
- 1.5 Unit (basic structure)
- 1.6 Nucleus (minimal core + bootstrap)
- 1.7 Console Unit (first human interface)
- 1.8 Make it interactive and test the full loop
- 1.9 Basic Naming support
- 1.10 Simple demo Units + first Improver experiments

Phase 2 — Persistent Object Substrate

Real durability, better compaction policies, crash recovery testing, and stronger guarantees that acknowledged versions survive process death.

Phase 3 — Minimal Interactive System

Polish the Console, add reliable naming and intent resolution, support birthing/killing Units and granting Capabilities fluidly, and demonstrate complete intent → execution loops.

Phase 4 — Capability & Security Hardening

Full token lifecycle, revocation, auditing, threat modeling, and making Capability UX feel natural.

Phase 5 — First Real Device Units & Human Interface Experiments

Simple device Units, early Studio / spatial interface prototypes, richer intent handling.

Phase 6 — Lowering Toward Metal

Re-implement critical paths in safer or lower-level languages, explore real hardware experiments.

Later / Parallel Tracks

- Distribution model (multi-machine)
- Formal verification of Nucleus properties
- Performance measurement framework
- Advanced Improver intelligence

PART 3 — HOW WE BUILD TOGETHER

This is the working agreement so we stay aligned.

Our Method

- We go step by step. No giant jumps.
- For each step I will tell you: the file name, exactly what code to put in it, and what to do next.
- You implement it (in Visual Studio or any editor you prefer).
- You run and test it.
- You tell me what happened (success, error messages, questions).
- Only then do we move to the next small step.

Tools

You can use Visual Studio, Visual Studio Code, or any Python environment you like. We will use Python 3 for Phase 1 because it is clear, fast to write, and easy to inspect. Later phases may introduce other languages when we move closer to the metal.

Pace

We move at a human pace. It is better to understand each piece deeply than to rush. You can always ask me to explain anything again or to break a step into even smaller pieces.

Starting Point

We will restart Phase 1 cleanly from the first step. I will begin by giving you the very first action: creating the project folder and the first file.

*This document is complete.
Save it. We will refer back to it often.*

— End of Design Document —